Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Dersin ilk yarısında sayı sistemleri, işaretli sayılar ve IEEE 754 floating point gösterimi işlenir. İkinci yarısında algoritma kavramına örneklerle devam edilir ve "iyi algoritma"nın ölçütleri konuşulur.
- Sayı sistemleri
  - Bilgisayar 0 ve 1 ile çalışır, ama 16, 32 veya 64 bitlik bir komutu ya da değeri ikilik sistemde okuyup anlamlandırmak zordur. Bu yüzden başka sayı sistemleri kullanılır.
  - İkilik (binary): rakamlar 0-1, taban 2.
  - Sekizlik (oktal): rakamlar 0-7, taban 8.
  - Onluk (desimal): rakamlar 0-9, taban 10.
  - On altılık (heksadesimal): rakamlar 0-9 ve A-F (A = 10, ..., F = 15), taban 16. Sayının altına 16 veya H yazılır.
- Basamak değeri
  - Onluk sistemde $385 = 3 \cdot 10^2 + 8 \cdot 10^1 + 5 \cdot 10^0$'dır.
  - Diğer sistemlerde de aynı mantık geçerlidir. Hangi tabandaysak o tabanın kuvvetleri kullanılır: $2^0, 2^1, 2^2, \ldots$ veya $8^0, 8^1, 8^2, \ldots$. Kuvvetler her zaman sağdan b^0 ile başlar.
  - Örnek: $(111)_2 = 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 7$.
- İkilikten sekizlik ve on altılık sisteme dönüşüm
  - İkilikten onluğa doğrudan dönüşüm uzun sayılarda zordur (2^{16}, 2^{17} gibi kuvvetler hesaplanır). İkilikten sekizlik veya on altılık sisteme geçmek çok daha kolaydır.
  - Sekizliğe geçerken bitler en düşük anlamlı bitten (LSB) başlayarak 3'erli gruplanır. Çünkü $2^3 = 8$'dir ve 3 bitle en fazla 7 yazılabilir.
  - On altılığa geçerken bitler LSB'den başlayarak 4'erli gruplanır. Çünkü $2^4 = 16$'dır ve 4 bitle en fazla 15 yazılabilir.
  - Gruplamaya MSB'den başlamak yanlıştır. Soldaki eksik bitlere 0 eklenebilir, sayının değeri değişmez (0385 = 385).
- Ters yönde dönüşüm
  - Bölme yöntemi: sayı sürekli 2'ye bölünür, kalanlar sondan başa doğru yazılır. Örneğin $13 \to 1101$.
  - Basamak değeri yöntemi: 4 bitlik bir grupta basamak değerleri 8, 4, 2, 1'dir. 13 içinde 8 var (1), kalan 5 içinde 4 var (1), kalan 1 içinde 2 yok (0), kalan 1 içinde 1 var (1). Sonuç: 1101. Hoca bu yöntemi önerir.
  - 0'dan 15'e kadar sayıların ikilik karşılıkları kullanıla kullanıla akılda kalmalıdır.
- Kesirli sayılar
  - İkilikten onluğa: noktanın solundaki basamaklar $2^0, 2^1, \ldots$, sağındakiler $2^{-1}, 2^{-2}, 2^{-3}, \ldots$ ile çarpılır. Örnek: $(1010.011)_2 = 8 + 2 + \frac{1}{4} + \frac{1}{8} = 10.375$.
  - Onluktan ikiliğe: kesirli kısım sürekli 2 ile çarpılır. Her çarpımın tam kısmı (0 veya 1) bir bit olur, kesirli kısımla devam edilir. Kesirli kısım 0 olunca durulur. Bitler yukarıdan aşağıya yazılır.
  - Kesirli kısım hiç 0'a ulaşmayabilir. Bu durumda belli bir yerde kesilir. Bu, Sayısal Analiz dersindeki kesme hatalarından biridir.
- İşaretli tam sayılar
  - Bitlerden biri işaret bitidir (1 negatif, 0 pozitif), geri kalanlar büyüklüğü tutar.
  - İşaret biti ortadaki bir bit olsaydı, bilgisayar sayıyı çözerken işaret bitini ayırıp iki parçayı birleştirmek (maskeleme) zorunda kalırdı. Bu yüzden işaret biti en anlamlı

bite (MSB) konur.
- Negatif sayılar: ikiye tümleme (two's complement)
  - Bir sayının negatifini bulmak için önce bire tümleyeni (one's complement) alınır: 0'lar 1, 1'ler 0 yapılır. Sonra 1 eklenir.
  - Kısa yol: sağdan başlayarak ilk 1'e kadar (o 1 dahil) bitler aynen yazılır, geri kalan bitler ters çevrilir.
  - Aynı işlem negatiften pozitife geçerken de kullanılır.
  - n bitle -2^{n-1} ile $2^{n-1}-1$ arasındaki sayılar yazılabilir. 4 bitle bu aralık -8 ile $+7$'dir.
- IEEE 754 floating point gösterimi
  - Bir reel sayı birçok şekilde yazılabilir (ör. $0.6875 = 6.875 \times 10^{-1}$). Herkesin aynı şekilde saklaması ve aynı hassasiyeti elde etmesi için bir standart gerekir. Bu standart IEEE 754'tür. Hoca, kablosuz iletişim standardı IEEE 802.11'i de benzer bir örnek olarak verir.
  - İki türü vardır: single precision (32 bit) ve double precision (64 bit). Precision hassasiyet demektir.
  - Bit dizisi üç alana bölünür: işaret (S), exponent ve fraction (mantissa).

	İşaret	Exponent	Fraction	Bias
Single (32 bit)	1 bit	8 bit	23 bit	$2^{8-1}-1=127$
Double (64 bit)	1 bit	11 bit	52 bit	$2^{11-1}-1=1023$

- Binary normalization
  - Sayı ikilik sisteme çevrilir ve bilimsel gösterimdeki gibi noktanın solunda tek bir basamak kalacak şekilde kaydırılır: $\pm 1.xxx \times 2^y$.
  - Bu yazımda noktanın solundaki basamak her zaman 1'dir. Bu yüzden saklanmaz. Fraction alanına yalnızca noktanın sağındaki bitler yazılır.
  - Exponent alanına $y +$ bias yazılır. Bias eklemek, üssü belli bir aralığa çeken bir normalizasyondur.
  - Saklanan bitlerden sayının değeri şöyle bulunur: $X = (-1)^S \times (1 + \text{fraction}) \times 2^{\text{exponent}-\text{bias}}$.
- Algoritma ve komut kümesi
  - Algoritma, belirli (unambiguous) ve sonlu (finite) sayıda komuttan oluşur ve bir problemi çözer. Bu problem bilgisayar problemi olmak zorunda değildir: hastanede COVID şüphelisine uygulanan prosedür, askeriyede bir cihazın kullanım talimatı da algoritmadır.
  - Hoca, ders kitabındaki origami örneğini gösterir: her sembol belli bir katlama işlemine karşılık gelir. Kağıttan kuş yapmak bu sınırlı ve tanımlı işlemlerin adım adım uygulanmasıyla olur.
  - Algoritma program olarak, sözlü anlatımla, akış diyagramıyla veya sözde kodla (pseudocode) ifade edilebilir.
- Üç temel yapının tekrarı
  - Sıralı (sequential): toplama, çıkarma, atama, karşılaştırma gibi değer üreten her işlem. == de bir sonuç (1/0) üretir, bu yüzden sıralı bir işlemdir.
  - Şartlı (conditional): `if` ve `if-else`. "Evde kekik yoksa yerine nane koy" bir if-else, "evde kekik yoksa hiçbir şey koyma" yalnızca if'tir.
  - Tekrarlı (repetitive): belirli tekrar `for`, belirsiz tekrar `while` / `do-while` (bazı dillerde `repeat-until`).
  - `a = b == c;` iki sıralı işlemdir: önce karşılaştırma yapılır (işlemcide compare komutu), sonra sonuç `a`'ya atanır. `if (b == c)` ise karşılaştırmanın sonucuna göre bir kısmın çalışıp diğerinin çalışmamasını sağlar.
- Yol tarifi örneği (Ataköy'den Esenler Otogarı'na)
  - Hiçbir şey bilmeyen bir turiste tarif yapılır: Marmaray girişine git, otomata para koy, düğmeye bas, kartı al, turnikeden geç, Marmaray gelince bin, Şirinevler'de in, metroya geç, Otogar'da in.
  - "Parası yoksa ATM'den para çek" bir şartlı yapıdır.
  - "Şirinevler'e gelene kadar seyahat et" bir `while`'dır. "Üç durak sonra in" bir `for`'dur. Seyahat en az bir durak süreceği için `do-while` ile de ifade edilebilir.
  - Hoca, Barcelona metrosundaki turnikenin videosunu gösterir: kart takılana kadar

bekler, kart alınana kadar açılmaz, geçiş olana kadar bekler. Hoca bu turnikeyi Ayrık Matematik dersinde sonlu durum makinesi (finite state machine) sorusu olarak sormuştur.

- Robota araba çizdirme etkinliği
  - Öğrenciler "kalemi al", "aşağı doğru çizgi çek", "dur", "sağa doğru çiz", "köşeyi merkez kabul ederek yuvarlak çiz" gibi komutlarla bir robota araba çizdirir.
  - Robot "dikdörtgen çiz" veya "başladığın noktaya git" komutlarını bilmez. Yalnızca tanımlı komutlar kullanılabilir.
  - Noktalara isim vermek (1, 2, ..., 9) tarifi kolaylaştırır.
- Tanımlı olmayan işlem kullanılamaz
  - Örneğin iki sayının EBOB'unu (GCD) bulan bir algoritmada `min(10, 80)` yazmak algoritma değildir. Bu bir fonksiyondur. İşlemcide minimum bulan bir komut yoksa minimumun nasıl bulunacağı tarif edilmelidir.
- İyi (kaliteli) algoritmanın ölçütleri (öğrencilerle birlikte çıkarıldı)
  - Planlı olması: önce kafada bir yaklaşım (approach) tasarlanır, sonra adımlara dökülür.
  - Doğruluk (correctness, accuracy): istenen sonucu vermesi. Kesin sonuca ulaşılamayan problemlerde optimum sonuç kabul edilebilir (ör. hava tahmininde %98 ile %99 doğruluk farkı).
  - Kaynakların verimli kullanımı: özellikle zaman (time) ve bellek (memory). Güç (power) de bir kaynaktır.
  - Netlik ve anlaşılabilirlik.
  - Kullanıcı dostu olması (user friendly). Bu daha çok arayüzle ilgilidir.
  - Genellenebilirlik: genel bir çözüm üreten algoritma, yalnızca özel bir duruma çözüm üretenden daha değerlidir.
- Ödünleşim (trade-off)
  - Ölçütlerin hepsi aynı anda en iyi olamaz. Probleme göre bazıları önceliklidir (problem-oriented approach).
  - Merge sort hızlıdır ama fazla bellek kullanır. Quick sort hem hızlı hem az bellek kullanır.
  - Gömülü sistemlerde bellek kısıtlıdır, bu yüzden bellek öne çıkar. Toplu işlemde (batch, ör. bir milyon kişiye mail göndermek) süre ikinci plandadır.
- İyi algoritma iteratif bir süreçle gelişir
  - "Science is approximation to the truth." Biri bir hata bulur, başkası düzeltir. Algoritmalar da böyle gelişir.
  - Bir algoritmanın daha iyi olduğunu iddia etmek için diğer algoritmaların ölçütleri (ör. selection sort, bubble sort) analiz edilir ve karşılaştırılır. En iyi ve en kötü durum analizleri ileride anlatılacaktır.
  - Hoca sıralama için "bir buçuk döngü" ifadesini kullanır. Bununla $n \log n$ karmaşıklığı kasteder. Karşılaştırmaya dayalı genel bir sıralamayı tek döngüyle yapmak bugüne kadar başarılamamıştır (counting sort gibi özel algoritmalar hariç).
  - Algoritma geliştirmenin bir algoritması yoktur. Algoritma geliştirerek, hata yaparak ve tecrübeyle öğrenilir. "C'yi biliyorum ama problemi nasıl çözeceğimi bilmiyorum" diyen kişide eksik olan algoritma tecrübesidir.
- 1'den N'e kadar toplam
  - Döngüyle çözüm doğrudur, ama işlem sayısı N ile büyür (yaklaşık $4N + 3$). Gauss formülü $\frac{N(N+1)}{2}$ ise N ne olursa olsun yaklaşık 4 işlemde sonucu verir ve `i` değişkenine de gerek bırakmaz.
  - Problem m'den n'e kadar toplama genişletilirse aritmetik dizi toplam formülü kullanılır. $-n$'den $+n$'e kadar toplam doğrudan 0'dır.
- Çözülmeyi bekleyen problemler ve şifreler
  - Google'ın PageRank algoritması, yüz tanıma, görüntülü görüşmede görüntü ve ses aktarımı gibi problemlerde daha iyi algoritmalar geliştirilerek öncekiler geride bırakılır (outperform).
  - 500 farklı karakterle 10 haneli bir şifre oluşturulursa 500^{10} olası şifre vardır. Kaba kuvvetle (brute force) çözmek için bunların hepsini denemek gerekir. Bu üstel (exponential) zamandır.
  - Siteler giriş deneme sayısını sınırlayarak ve CAPTCHA kullanarak kaba kuvvet saldırılarını engeller.
  - Şifreler veritabanında açık metin olarak değil, geri döndürülemeyen bir hash fonksi-

yonundan geçirilerek saklanmalıdır. Girişte girilen şifre yeniden hash'lenir ve saklanan değerle karşılaştırılır. "Şifremi unuttum" deyince şifrenizi size mail ile gönderen bir site, şifreyi güvenli saklamıyor demektir. MD5 gibi bazı eski hash fonksiyonları kaba kuvvetle kırılabilmektedir.
  - Problemler polinom zamanda çözülebilenler ve çözümü henüz polinom zamanda bulunamamış olanlar (NP) olarak sınıflandırılır. Bugün çözülemeyen bir problem birkaç yıl sonra çözülebilir.

Hocanın Özellikle Vurguladığı Kısımlar

- İkilikten sekizlik veya on altılık sisteme geçerken gruplamaya her zaman en düşük anlamlı bitten (LSB, sağdan) başlanır. Hoca dördüncü sınıfta bile MSB'den gruplayan öğrenciler gördüğünü söyler.
- İkilik bir sayıda 0 ve 1'den başka rakam olamaz. Kesirli dönüşümlerde basamağa 2 yazan öğrenciler olur.
- 0-15 arası sayıların ikilik karşılıkları bir bilgisayar mühendisliği öğrencisinin aklına hemen gelmelidir.
- Algoritmada her işlem tanımlı olmalıdır. Robota "dikdörtgen çiz" diyemezsiniz, çizginin nasıl çekileceğini bilmesi gerekir. Aynı şekilde algoritmada `min()` gibi tanımsız bir işlem kullanılamaz.
- Değer üreten her işlem (== dahil) sıralı bir yapıdır. Şartlı yapı yalnızca `if` ile oluşur.
- Algoritmada dördüncü bir yapı yoktur.
- İyi algoritmanın ölçütleri arasında her zaman bir trade-off vardır. Hangi ölçütün öne çıkacağını problem belirler.
- Algoritma ancak algoritma geliştirerek öğrenilir. code.org'daki problemleri ve Fatih Hoca'nın sayfasındaki ders 1 sorularını çözün. Bir sonraki derste bu sorular gruplarla tartışılacaktır.
- Ders kitabı: J. Glenn Brookshear, "Computer Science: An Overview" (13. baskı, 2019). Hoca İngilizcesini okumayı önerir. Slaytlar yalnızca konu başlıklarıdır, kitap okunmalıdır.

Kısa Tekrar Notları

- Taban 2: 0-1. Taban 8: 0-7. Taban 10: 0-9. Taban 16: 0-9, A-F.
- İkilikten sekizliğe: LSB'den 3'erli grupla. On altılığa: 4'erli grupla.
- Onluktan ikiliğe (tam kısım): 2'ye böl, kalanları sondan başa yaz.
- Onluktan ikiliğe (kesirli kısım): 2 ile çarp, tam kısımları baştan sona yaz.
- Ondalık basamaklar: $2^{-1} = 0.5$, $2^{-2} = 0.25$, $2^{-3} = 0.125$, $2^{-4} = 0.0625$.
- İkiye tümleme: bitleri ters çevir, 1 ekle. Kısa yol: sağdan ilk 1'e kadar aynen yaz, gerisini ters çevir.
- n bit işaretli aralık: -2^{n-1} ile $2^{n-1} - 1$.
- IEEE 754 single: 1 + 8 + 23 bit, bias 127. Double: 1 + 11 + 52 bit, bias 1023.
- $X = (-1)^S \times (1 + \text{fraction}) \times 2^{E-\text{bias}}$.
- Normalize edilmiş sayı her zaman $1.xxx$ ile başlar. Baştaki 1 saklanmaz.
- İyi algoritma: planlı, doğru, verimli (zaman/bellek), net, kullanıcı dostu, genellenebilir.
- $1 + 2 + \cdots + N = \frac{N(N+1)}{2}$.

Detaylı Açıklamalar

Ders, Telegram grubuyla ilgili duyurularla başlar. Hoca grupta herkesin kendi adıyla bulunmasını ister. Grubun tek yönlü bir kanal yerine herkesin yazabildiği bir grup olmasını tercih ettiğini söyler ve bu arada simplex, half duplex ve full duplex iletişimi örnekler (televizyon, telsiz, telefon). Ardından OneNote'ta dersin konularına geçer.

Dönüşüm örneği

Derste bir ikilik sayı sekizlik ve on altılık sisteme öğrencilerle birlikte çevrildi:

```
İkilik (3'erli):   010 101 110 100 110 111
Sekizlik:           2   5   6   4   6   7      → (256467)_8
```

```
İkilik (4'erli):   0001 0101 1101 0011 0111
On altılık:          1    5    D    3    7     → (15D37)_16
```

Onluk karşılığı derste hesaplanmadı. Kontrol için: $(15D37)_{16} = 89399$.

Ters yönde, D'nin ikilik karşılığı basamak değerleriyle bulunur: $13 = 8 + 4 + 1 \rightarrow 1101$.

Kesirli sayı örnekleri

$(1010.011)_2$'nin onluk karşılığı:

$$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 8 + 2 + \tfrac{1}{4} + \tfrac{1}{8} = 10.375$$

$(0.6875)_{10}$'un ikilik karşılığı:

```
0.6875 × 2 = 1.375   → 1   (devam: 0.375)
0.375  × 2 = 0.75    → 0   (devam: 0.75)
0.75   × 2 = 1.5     → 1   (devam: 0.5)
0.5    × 2 = 1.0     → 1   (kesirli kısım 0, dur)
```

Sonuç: $(0.1011)_2$. Sağlaması: $\frac{1}{2} + \frac{1}{8} + \frac{1}{16} = \frac{8+2+1}{16} = \frac{11}{16} = 0.6875$.

İkiye tümleme örneği

8 bitte +3'ten −3'e geçiş:

```
+3                  : 0000 0011
Bire tümleyen       : 1111 1100
+1                  : 1111 1101   → -3
```

Kısa yolla: sağdan ilk 1 (en sağdaki bit) aynen yazılır, geri kalan bütün bitler ters çevrilir. Sonuç yine `11111101` olur.

IEEE 754 örneği: 6.375'i single precision ile saklamak

1. İkilik sisteme çevir: $6 = 110$, $0.375 = 0.011$. Yani $6.375 = (110.011)_2$.
2. Normalize et: nokta iki basamak sola kaydırılır. $110.011 = 1.10011 \times 2^2$.
3. İşaret: sayı pozitif, $S = 0$.
4. Exponent: $2 + 127 = 129 = (10000001)_2$.
5. Fraction: noktanın sağındaki bitler, 23 bite 0 ile tamamlanır: `10011000000000000000000`.

```
S   Exponent (8)   Fraction (23)
0   10000001       10011000000000000000000
```

Geri dönüş: $(-1)^0 \times (1 + 0.59375) \times 2^{129-127} = 1.59375 \times 4 = 6.375$.

Fraction alanı 23 bitle sınırlıdır. Kesirli kısmı sonsuza giden sayılarda (ör. 0.333...) 23 bitten sonrası kesilir ve kalan kısım hata olarak kalır. Hoca aynı örneği double precision ile denemeyi ödev olarak bırakır.

IEEE 754 örneği: bitlerden sayıya

```
S   Exponent (8)   Fraction (23)
1   10000001       01000000000000000000000
```

- $S = 1$, sayı negatif.
- Exponent $= 129$, gerçek üs $129 - 127 = 2$.
- Fraction $= 0.01_2 = 2^{-2} = 0.25$.
- $X = (-1)^1 \times (1 + 0.25) \times 2^2 = -1 \times 1.25 \times 4 = -5.0$.

Hoca çalışma için 2.66 ve 3.66 gibi sayıları IEEE 754 formatına çevirmeyi önerir.

1'den N'e kadar toplamın işlem sayısı

```
int i = 0;                  /* 1 işlem */
int tp = 0;                 /* 1 işlem */
for (i = 1; i <= N; i++)    /* i = 1: 1 işlem, i <= N: N, i++: N */
    tp = tp + i;            /* toplama N, atama N */
```

Hoca toplamı $4N + 3$ olarak sayar. N = 100 için yaklaşık 403 işlem yapılır. Döngü bittiğinde i'nin değeri $N + 1$'dir (N = 10 için 11). `i = 1` ataması tanımlamaya taşınırsa (`int i = 1;` ve `for (; i <= N; i++)`) bir işlem azalır: $4N + 2$.

Gauss formülüyle aynı sonuç sabit sayıda işlemle bulunur: `tp = N * (N + 1) / 2;` (bir toplama, bir çarpma, bir bölme, bir atama). N ister 100, ister bir milyon olsun işlem sayısı 4'tür. Ayrıca `i` değişkenine gerek kalmadığı için bellek kullanımı da azalır (3 değişken yerine 2).

Ders sonrasında iki yöntem `gcc -Wall` ile derlenip denendi. N = 1, 10, 100, 1000 için iki yöntem aynı sonucu verdi (1, 55, 5050, 500500). Döngü sonunda i'nin değeri her seferinde N + 1 oldu.

Ders sonrası notlar

- **IEEE 754'te baştaki 0:** Bir öğrenci noktanın solundaki basamağın 0 olabileceğini sordu. Hoca 0.110011×2^3 yazımıyla exponent'i $3+127$ yapıp fraction'a `110011` yazmayı önerdi ve bunun yine 6.375 vereceğini düşündüğünü söyledi. Bu doğru değildir. Standart, normalize edilmiş sayılarda noktanın solundaki basamağın her zaman 1 olmasını şart koşar ve bu 1 saklanmaz. Önerilen bitler formüle konursa $1.110011_2 \times 2^4 = 28.75$ elde edilir. Hocanın biraz sonra söylediği doğrudur: sıfırdan farklı her sayı normalize edildiğinde başta 1 kalır. 1'den küçük sayılar da negatif üsle normalize edilir (ör. $0.6875 = 1.011_2 \times 2^{-1}$). Başta 0 bulunan tek durum, exponent alanının tamamen 0 olduğu çok küçük sayılardır (subnormal sayılar).
- **İşlem sayısı:** Derste $4N + 3$ hesaplandı. `i <= N` karşılaştırması aslında $N + 1$ kez yapılır, çünkü döngüden çıkmak için son bir kez (yanlış sonuç veren) kontrol yapılır. Bu sayılırsa toplam $4N + 4$ olur. Bu fark sonucu değiştirmez: işlem sayısı N ile doğrusal büyür.
- **m'den n'e kadar toplam:** Derste aralık negatif ve pozitif sayıları içerdiğinde "doğrudan m − n yaparım" denildi. Genel formül her durumda geçerlidir: $\sum_{k=m}^{n} k = \frac{(n-m+1)(m+n)}{2}$. Örneğin -3'ten 5'e kadar toplam $\frac{9 \cdot 2}{2} = 9$'dur (-3 ile 3 birbirini götürür, $4 + 5 = 9$ kalır).
- **Precision ve recall:** Hoca "precision"ın hassasiyet anlamına geldiğini anlatırken makine öğrenmesindeki precision ve recall metriklerini COVID testi örneğiyle açıkladı. Recall: gerçekten hasta olanların kaçının pozitif işaretlendiği. Precision: pozitif işaretlenenlerin kaçının gerçekten hasta olduğu. IEEE 754'teki single ve double precision ise bu metriklerle ilgili değildir. Burada precision, sayının kaç anlamlı bitle saklandığını belirtir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.